Teaching Philosophy and Support of Diverse Learners

Waseem Raza, PhD | Computer Science and Software Engineering

Helping students reason about programs

In a programming class, a correct output is useful but incomplete evidence of understanding. I want students to explain the problem being solved, trace how the program reaches its result, test unexpected inputs, and describe what they would change if the requirements grew. Those habits are important in an introductory Python course and in an advanced software project. They also bridge the subjects I taught earlier and the software-intensive research and engineering work I do today.

I taught Computer Programming, Communication Systems, and Wireless Communication as a Lecturer in the University of Lahore's Department of Technology. I supported communication laboratories at UET Taxila and mentored students working on computing-intensive research at COMSATS. Through additional tutoring of students at the tenth- and twelfth-grade levels, I have seen that students bring very different assumptions about how to approach a technical problem. It helps to make the hidden steps explicit, from reading a problem statement through testing a solution.

My primary areas of teaching preparation are programming, data-oriented computing, computer and communication networks, and applied machine learning. I use Python and C/C++, Git and Linux, scientific computing, and ML frameworks in research and industry work. I would also contribute to departmental curriculum needs after appropriate preparation; however, I would not claim to have previously served as instructor of record for algorithms, Java-based software design, operating systems, cybersecurity, or other courses that are not documented in my teaching record.

Building foundations through testing and iteration

For an introductory exercise, I might ask students to predict how a function behaves on an empty input, a duplicate value, and a malformed record before they run it. Once they have checked their prediction, they would write a few tests and refactor the function for clarity. Later, students could compare two approaches to searching or processing data and explain differences in time, memory, and maintainability. Each stage moves from syntax toward design judgment.

Advanced assignments can use small, real datasets to teach reproducibility and responsible interpretation. Students might build a simple anomaly detector for network measurements, evaluate a classifier on held-out subjects, or create a lightweight dashboard to explain failure cases. The emphasis would be on clear baselines, appropriate data splits, documentation, and honest reporting. These would be proposed exercises tailored to departmental course outcomes, not descriptions of courses I have already taught.

My experience in industrial telecommunications and ML offers realistic examples of requirements changing, incomplete data, and software needing to survive beyond the first demonstration. Yet I would keep course prerequisites and core CS ideas in view; an AI application should never be a shortcut around data structures, algorithmic reasoning, or disciplined software design.

Assessment and an inclusive learning environment

I would align coding tasks, quizzes, code review, and project rubrics with observable outcomes. For example, the assessment of a programming submission should distinguish functional correctness, decomposition, testing, error handling, readability, and explanation. Frequent small exercises can help identify a misconception before it becomes embedded in a large project. I would provide examples of constructive code review and encourage students to compare solutions without reducing the discussion to who wrote the shortest program.

Supporting diverse learners starts with predictable course structure: accessible material, explicit learning goals, clear software setup instructions, and feedback that can be acted upon. Some students arrive with years of programming experience; others are new to debugging tools. I would use short prerequisite refreshers and rotating group responsibilities so that all students practice the essential technical decisions. Structured office hours, flexible ways to ask questions, and

collaboration with teaching-support and accessibility staff can maintain high expectations while reducing avoidable barriers.

Computer science instruction also benefits from discussing the people affected by technical systems. When teaching ML or data analysis, I would ask students to examine data provenance, differences in errors across groups, and privacy concerns. In software projects, I would emphasize documentation and accessibility because other people will have to use and maintain what students build.

Mentoring and assessment of my own teaching

For undergraduate projects I would identify the smallest meaningful deliverable, establish a baseline and test plan, and meet regularly as students work toward independent decisions. A student should have an opportunity to explain a failure and revise a design, not simply accumulate features. I would use patterns in student work, midsemester feedback, and conversations with colleagues to improve course pacing and assignments. My goal is for students to become capable programmers and careful evaluators of their own work.